Reset Early, Reset Often, Saturate the SAT Core

Martin S. Fixman[1], Fredrik Rømming[2], and Sean B. Holden[3]

University of Cambridge,
United Kingdom
[1]msf43@cl.cam.ac.uk [2]fr409@cl.cam.ac.uk [3]sbh11@cl.cam.ac.uk

Abstract

Connection tableau provers usually try to close one tableau before moving on to another branch of the search. We introduce SATResetCoP, a variant of SATCoP that instead treats the accumulated SAT abstraction as the main search object: it repeatedly resets the active tableau and uses the new ground clauses produced by these attempts to drive the SAT abstraction toward contradiction. This drives tableau provers with a SAT nucleus closer to saturation provers such as Vampire. We compare this against implementations of leanCoP and SATCoP in the same backend framework. SATResetCoP solves 2971 TPTP problems under limits of 120 seconds and 1 million steps, an improvement of 54% over our implementation of leanCoP and 18% over our improved implementation of SATCoP. These results suggest that changing tableau guidance to force faster saturation of the ground SAT core improves the performance of teh prover.

1 Introduction

Connection tableau provers are attractive because the core proof search is simple and goal-directed.

They originated from semantic tableau refutation models proposed by Evert Beth and developed by Raymond Smullyan[3], which later added goal-directed guidance by Loveland[10].

Beckert and Posegga introduces first Prolog implementation of this model, leanTAP[2], which showed how these models could be implemented in small and swift programs that would make experimentation easier. This was the baseline used by Otten and Bibel to develop the first version of leanCoP

This is especially visible in leanCoP, where a competitive first-order prover can be implemented with very little machinery[14, 13]. The cost is that the prover has to make many local control decisions: which clause to start from, which connection to try next, when to backtrack, and when to restart.

These choices matter because connection proof search is not confluent. A locally valid connection can bind variables in a way that makes another branch impossible to close later. leanCoP's `cut` option shows that even a crude restriction on backtracking can make a large practical difference, although it loses completeness unless it is used inside a complete schedule[12].

SATCoP changes the objective slightly. Instead of using the tableau only as a proof object that must eventually close, it also uses the tableau as a source of ground clauses for an incremental SAT solver[19]. Once the accumulated ground clauses are propositionally unsatisfiable, the prover can stop even if the current first-order tableau has not closed. This suggests a different view of tableau control: a good run is not only one that closes the tableau quickly, but one that generates useful ground instances quickly.

This paper explores that view by resetting the tableau aggressively. If the current bounded search stops adding new ground information to the SAT solver, we abandon it and start another search. This is incomplete as a standalone proof procedure, but useful when the goal is to saturate the ground SAT clause set fast. The rest of the paper describes the resulting prover,

SATResetCoP, and compares it against leanCoP-style and SATCoP-style baselines on TPTP problems.

Although the implementation studied here is not an extension of Vampire, the question is closely aligned with the themes of the Vampire workshop. The method studies how a first-order prover can combine goal-directed proof search with a persistent SAT abstraction, and how search-control choices affect the rate at which useful ground instances are produced. The experiments are therefore intended as a small, controlled study of prover strategy design and SAT-assisted first-order reasoning, rather than as a replacement for saturation-based systems such as Vampire.

2 Preliminaries

2.1 Connection Calculus and Tableau Solving Methods

Connection tableaux and model elimination give a goal-directed alternative to the saturation style used by provers such as Vampire[20]. Instead of repeatedly deriving clauses from a global clause set until a contradiction is found or the search space saturates[1, 23], connection calculi try to close a tableau by connecting goal literals to complementary literals, either on the current path or in a fresh copy of an input clause[9, 15].

The standard clausal connection tableau has three proof-building inferences[9, 16].

Start Choose an input clause and make its literals open goals.

Extension Open a branch on a clause containing a complementary literal, and add the remaining literals as new subgoals. This might require a substitution of variables with terms.

Reduction Closes the current goal against a complementary literal on the branch. Like the previous step, this might require adding a substitution.

The calculus is sound and complete with backtracking and iterative deepening, but it is not confluent: a locally valid connection can bind variables in a way that prevents later branches from closing[9].

Following the usual accounting for connection-tableau provers, we use *step* to mean one of the three proof-building inferences above. Control operations such as backtracking, cuts, and resets affect which proof-building inferences are attempted, but are not themselves counted as steps.

Many different connection tableau provers have been developed over the years. leanCoP adapted this style of connection tableau proving to Prolog and became the basis of a large family of provers for classical and non-classical logics[14, 13].

leanCoP comes with several optional predicates. Most importantly, `cut` commits a Prolog cut `(!)` after every successful Reduction and Extension. This removes many alternative local continuations and makes the base prover incomplete, but it also converges to a solution considerably faster[12]. Additionally, leanCoP iteratively increases the maximum tableau depth when the current options are saturated.

To keep the prover complete, the `comp(I)` option removes the maximum height restrictions and the `cut` when starting a tree that would have had maximum height `I`.

Later systems can be read as attempts to make this local control less crude. meanCoP separates cut on reductions from cut on extensions and introduces exclusive cuts, which preserve more alternatives than the inclusive cuts used by leanCoP[6]. hopCoP takes the opposite route: instead of deliberately throwing away alternatives, it learns constraints explaining why a search

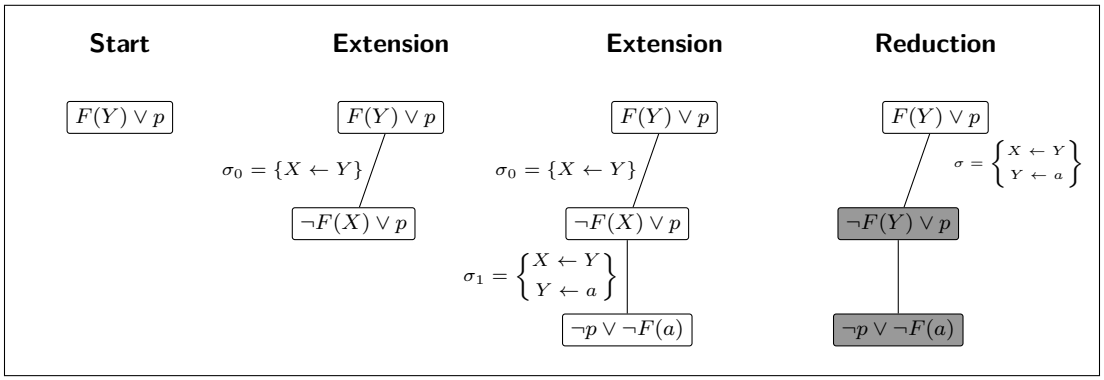


Figure 1: The three proof-building steps of a clausal connection tableau, taken from the tableau closure of SYN052-1.p[17]. The two Extension steps add an extra clause to the tableau; the Reduction step closes the nodes against a complementary literal already on the path. A full solution can, in several different strategies, be found in Appendix A.

state is stuck, thereby reducing repeated backtracking while preserving completeness at a fixed iterative-deepening level[18].

Functional connection provers, mlCoP, and monteCoP reorder proof steps using learned information or Monte Carlo Tree Search[7]. rlCoP and plCoP make this more explicit by treating proof search as a learning-guided control process over leanCoP-style actions[8, 26, 27]. The common theme is the one we use below: connection-tableau proving is naturally viewed as a sequence of small control decisions over proof actions, cuts, backtracking, and restarts.

2.2 SATCoP and Methods that use SAT Saturation in a Tableau

SAT and SMT solvers have long been used to support first-order reasoning[19, 22]. For a connection prover, the useful observation is simple: every clause that appears in a tableau is an instance of an input clause. If we ground these tableau clauses and collect them in a propositional SAT solver, then a SAT refutation of the collected ground clauses is a sound refutation of the original first-order problem.

Rawson and Reger implement this idea in SATCoP, a clausal connection-tableau prover based on leanCoP[19].

During backtracking search, clauses present in the tableau are heuristically grounded and added to an incremental SAT solver. If the SAT solver reports that the accumulated ground clauses are unsatisfiable, proof search can stop even if the first-order tableau itself has not been closed[19]. The remaining tableau variables must be mapped to ground terms before a clause is sent to SAT. Their implementation follows a Schulz-style heuristic: use the most frequent constant in the conjecture, and fall back to a fresh constant if no conjecture constant is available[19, 22, § 4.2].

Later modifications to SATCoP gave it control of the tableau using results from the SAT solver. In a later note included in the same paper, Rawson and Reger also describe a literal-selection refinement: prefer extension choices whose selected literal is assigned true by the current SAT solver assignment[19, § 9.1].

The strategy used in this paper, SATResetCoP, pushes the ideas from SATCoP further by resetting trees as soon as the ground SAT clause set stops growing.

3 Research Vehicle

3.1 SATResetCoP Specification

We propose a new tableau prover based on SATCoP, but guided by the goal of saturating the SAT solver rather than closing the current tableau.

Successive experimentation led to a simple strategy for achieving this: aggressively reset the tree when the SAT solver stops receiving new ground clauses. This adds a broader set of ground clauses to the solver in less time, allowing the SAT solver to reach unsatisfiability earlier.

SATResetCoP uses many of the properties of SATCoP described in the original paper and in its "Future Research" section[19, § 9.1], but changes the tableau guidance to prioritise aggressive resets.

Just like PySATCoP, this approach is sound. While it is less complete than PySATCoP and other leanCoP variants, the speed at which it proves unsatisfiability can increase the number of problems solved within fixed time and step limits. It might be useful to incorporate SATResetCoP into the initial steps of a more complete schedule.

A concrete example of how PyLeanCoP, PySATCoP, and SATResetCoP handle SYN052-1.p can be found in Appendix A.

3.2 Implementation Details

Like SATCoP, the implementation keeps a persistent SAT solver alongside the active tableau search. The SAT state is not recreated during ordinary proof search. Ground clauses accumulated in earlier tableau attempts remain available in later attempts, and new clauses are added incrementally to the same solver instance.

The SAT state is implemented using CaDiCaL[4]. A new virtual operation, Reset, clears the tableau, the current substitution, and the proof sequence, and starts a new tableau attempt with a specified depth bound. It does not clear the persistent SAT solver: every reset starts from an empty tree, while the SAT solver keeps the ground information produced by earlier attempts. An example of how this operation works can be seen in Figure 2.

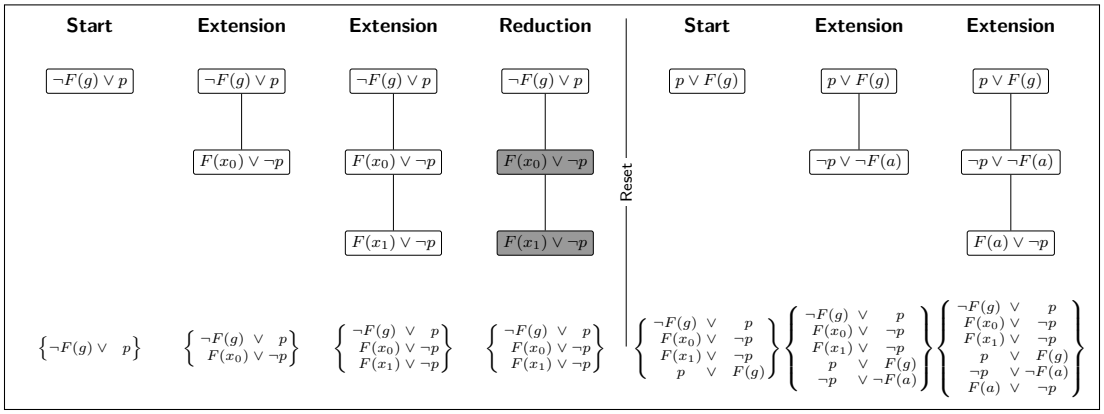


Figure 2: Example of SATResetCoP on SYN052-1.p. SATCoP and previous provers would backtrack after closing the branch with $F(x_1) \vee \neg p$; instead, SATResetCoP resets into a new tree. This grows the UNSAT core shown at the bottom faster than in SATCoP, allowing the SAT solver to reach unsatisfiability earlier. Substitutions are omitted for simplicity.

The grounded clauses used by PySATCoP and SATResetCoP come from clauses that have appeared in the tableau, after applying the tableau substitution accumulated so far. Start clauses are added to the SAT state when the run is initialised. After successful Extension and Reduction steps, the implementation grounds the clauses currently present in the tableau and records the genuinely new ones, which are later flushed to the SAT solver.

The grounding scheme follows the soundness argument of Section 2.2: the SAT solver receives only ground instances of clauses generated by the tableau. Ground terms are kept as they are, and variables assigned by the current tableau substitution are replaced by their assigned terms. Only tableau variables that remain unassigned need to be replaced by a ground term. Rawson and Reger follow the technique by Schulz[22] and use the most frequent constant in the conjecture, with a fresh constant as a fallback when the conjecture contains no constants[19, § 4.2]. Our Python SATCoP-style variants use the simpler fallback-style choice throughout, mapping every remaining tableau variable to the same fresh artificial constant. Thus PySATCoP$_0$, PySATCoP, and SATResetCoP all use a simpler grounding scheme than the original SATCoP implementation, while keeping the same soundness condition.

The reset policy does not affect the soundness of the SAT stopping condition. Every clause added to the SAT solver is a ground instance of an input clause, after applying the current tableau substitution and replacing any remaining tableau variables by the grounding constant. Therefore, if the accumulated propositional clauses are unsatisfiable, the original first-order clause set is unsatisfiable. The cuts and resets only affect which ground instances are discovered, and hence the completeness and performance of the search, not the soundness of a SAT refutation once found.

The reset policy is the main implementation difference between SATResetCoP and PySATCoP. At a dead end, the prover first asks whether the current tableau attempt produced any new ground clauses that have not yet been added to the SAT solver. If it did, those clauses are flushed to the SAT solver and the prover resets at the same depth. This gives the next tableau attempt the same depth bound, but a larger SAT state and a fresh randomised ordering of choices. If the attempt did not produce any new ground clauses, the prover treats the current depth as exhausted and resets at the next depth instead of spending more steps on local backtracking. The process stops as soon as the accumulated ground clauses become propositionally unsatisfiable.

SATResetCoP randomises the Start and Extension steps in the same way as SATCoP. When starting a new tree, the implementation shuffles the possible starts and the literals inside each start clause. When several extensions are possible from the leftmost open node, it shuffles the extension choices and the literal order inside the copied extension clause. Reductions are not randomised in the current implementation. The random generator is seeded, so the experiments remain reproducible.

Pseudocode for this implementation can be found in Listing 1.

3.3 Python Implementation

As a testbed for our experiments we use the Connections Prover by Rømming et. al[21], which provides primitives for the implementation of connection tableau provers. The base prover can be found in `https://github.com/fredrrom/connections`.

The connections prover makes it easy to implement different strategies for guiding the tableau proof. This keeps the SATResetCoP implementation simple, readable, and easy to evaluate.

The source code for this implementation is available at `https://github.com/mfixman/resetsatcop`, and is released under the GNU Lesser General Public Licence[5].

```
def solve_tableau(start_clauses):
  solver := incremental_sat_solver()
  solver.add(ground(start_clauses))
  pending := empty_set

  max_depth := 1
  tableau := empty_tableau(max_depth)

  while solve_sat(solver) != UNSAT:
    model := current_model(solver)
    actions := legal_actions(tableau)

    if any Start, Extension, Reduction in actions:
      model_rank(a) :=
        2 if a is Start
        0 if selected_literal(a) is true in model
        1 if selected_literal(a) is unknown in model
        3 if selected_literal(a) is false in model

      next_action = argmin(actions, key = model_rank)
      tableau_step(next_action)

      if next_action is Reduction or Extension:
        pending.add(ground(tableau))

    else:
      if pending is not empty:
        solver.add_all(pending)
        pending := empty_set
        tableau := empty_tableau(max_depth)
      else:
        max_depth := max_depth + 1
        tableau := empty_tableau(max_depth)
```

Listing 1: Pseudocode of the SATResetCoP control policy.

3.4 TPTP Benchmarking

Along with our implementation of SATResetCoP, the connections prover allows us to reimplement PyLeanCoP and PySATCoP in the same backend[21]. This provides a fairer baseline for comparison.

We run SATResetCoP, our two connection-tableau baselines, and a simple strategy that chooses the first possible action on 7815 CNF problems and 8779 FOF problems from the Thousands of Problems for Theorem Provers (TPTP) library[24]. These problems cover all 50 TPTP categories containing at least one CNF or FOF problem.

The library provides a varied problem set that is a standard benchmark for first-order provers. It also provides declared statuses for these problems, which are a useful sanity check for false unsatisfiability results.

4 Experimental Evaluation

4.1 Experiment Design

The connections prover includes PyCoP, a simple control policy that chooses the first legal non-backtrack action at each point and resets the tree with an increased maximum height when the current tree saturates. This serves as a baseline and sanity check. As a second control policy we use our implementation of leanCoP[12], PyLeanCoP. This is a Python version of the

leanCoP Prolog implementation, with the two recommended predicates used in the first step of the schedule by Otten[12, § 5.5].

`cut` Choose one extension or reduction and commit to it, similarly to Prolog's `!` operator.

`comp(7)` The prover increases the depth bound by 1 whenever the current tree saturates, until reaching maximum height 7. Afterwards, the height limit is removed.

These simple policies tend to be very effective in tableau provers[13].

Strategy	Explanation	Cut	SAT Refutation	SAT Guidance	Reset on Backtrack
PyCoP PyLeanCoP	Simple control policy leanCoP with initial set of predicates.	✓			
PySATCoP$_0$	Based on the initial implementation of SATCoP[19].	✓	✓		
PySATCoP	Improvements on SATCoP[19, § 9.1].	✓	✓	✓	
SATResetCoP	Novel strategy, specified in Section 3.1.	✓	✓	✓	✓

Table 1: The four control strategies and the test strategy used in this experiment.

The benchmark set contains 7815 CNF problems and 8779 FOF problems spread across 50 categories[24]. Each problem-strategy pair is run for at most 120 seconds and 1 million steps, on a single core of the University of Cambridge CSD3 Cluster with dual-socket Intel Xeon Platinum 8368Q CPUs at 2.60 GHz, with 38 cores per socket and one thread per core, and 3370 MiB RAM per core[25].

4.2 Results

4.2.1 Individual Results

SATResetCoP considerably increases the number of problems solved by both PySATCoP$_0$ and PySATCoP under the 120-second time limit, as shown in Table 2. The main driver is that the SAT solver reaches unsatisfiability in less time and fewer steps than in the previous strategies, as seen in Figures 3a and 3b. For the SAT-based variants, the 1 000 000 step limit is rarely reached, and most unsolved problems hit the 120 second limit instead.

The improvement is visible in both problem formats. SATResetCoP solves 462 more problems than PySATCoP, with 206 additional CNF problems and 256 additional FOF problems. Compared with PySATCoP$_0$, it solves 1100 more problems, an increase of 58.8%, and compared with PyLeanCoP it solves 1048 more problems, an increase of 54.5%.

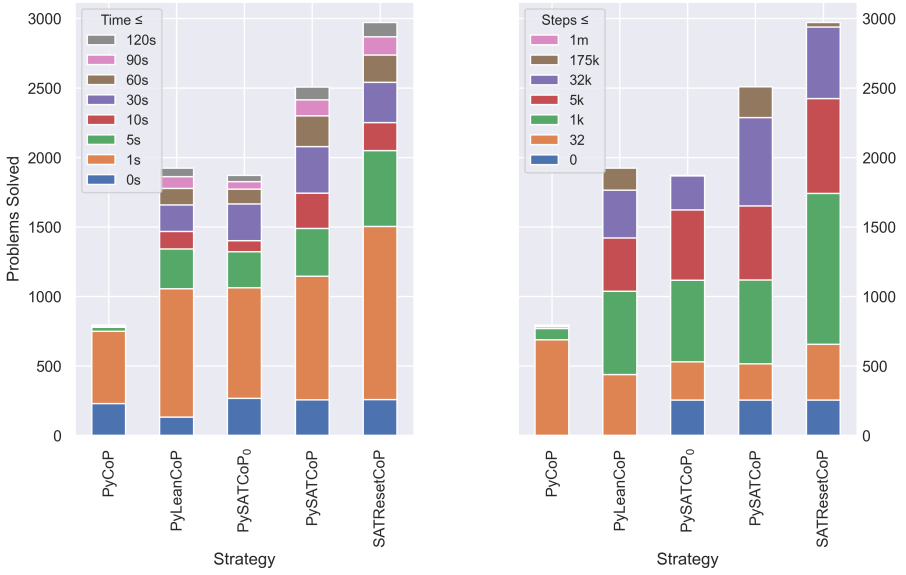
4.2.2 Comparative Results

The SAT guidance in SATResetCoP allows it to solve several problems left unsolved by strategies without the same SAT saturation mechanism, and even by PySATCoP$_0$ and PySATCoP, which use less aggressive guidance. A comparison of newly solved and newly unsolved problems can be seen in Table 3.

Figure 4 shows the number of problems solved by one strategy but not by another.

	CNF		FOF		Total	
strategy	Number	%	Number	%	Number	%
PyCoP	208	2.7	587	6.7	795	4.8
PyLeanCoP	723	9.3	1200	13.7	1923	11.6
PySATCoP$_0$	804	10.3	1067	12.2	1871	11.3
PySATCoP	1067	13.7	1442	16.4	2509	15.1
SATResetCoP	1273	16.3	1698	19.3	2971	17.9

Table 2: Number and percentage of CNF and FOF problems solved within the time limit by each strategy. Each strategy component, other than PySATCoP$_0$ the original version of SATCoP improves results on both CNF and FOF problems.



(a) Problems solved, stacked by time in seconds.

(b) Problems solved, stacked by number of steps. SAT-based strategies can solve a problem in 0 steps if the initial ground SAT clause set is already unsatisfiable.

Figure 3: Results of running the strategies on all problems under limits of 120 seconds and 1 million steps.

The improvement between SATResetCoP and the best previous experimental strategy, our PySATCoP implementation of SATCoP, is especially visible in TPTP categories where propositional contradictions are reached quickly in the accumulated ground abstraction, such as SYN,

Strategy	Newly Solved	Newly Unsolved	Total Solved
PyCoP			795
PyLeanCoP	1246	118	1923
PySATCoP$_0$	327	379	1871
PySATCoP	661	23	2509
SATResetCoP	598	136	2971

Table 3: Problems newly solved and newly unsolved by each strategy compared with the previous strategy in the table, together with the total number of solved problems.

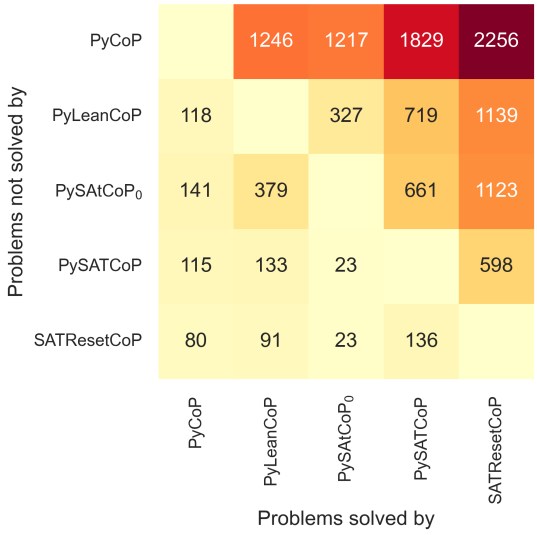


Figure 4: Pairwise comparison of strategy results: the cell in column y and row x shows the number of problems solved by strategy y, but not by strategy x. For example, SATResetCoP solved 1123 TPTP problems that PySATCoP$_0$ did not.

Strategy	SYN	SWC	SWV	GEO	NUM	Rest	Total
PyCoP	231	30	129	28	9	368	795
PyLeanCoP	486	14	163	216	86	958	1923
PySATCoP$_0$	459	2	161	177	90	982	1871
PySATCoP	549	11	173	260	151	1365	2509
SATResetCoP	672	64	215	293	179	1548	2971

Table 4: Number of problems solved, per category, for each strategy.

SWC, and SWV. As expected, this strategy also solves problems in fewer steps, as can be seen in Figure 5.

Appendix A shows the solution to the TPTP problem SYN052-1[17] in three different strategies. This is an example of the kind of problem on which SATResetCoP performs best, since it can be solved with a relatively small UNSAT core.

4.2.3 Soundness Verification: Problem Satisfiability

The TPTP library includes declared statuses for its problems[24]. While this does not demonstrate soundness, it provides a useful check that the strategy has not become unsound by mistake.

We make a second run of SATResetCoP with a longer 10-minute limit, whose results can be seen in Table 5. The results are consistent with what we would expect if the prover were sound, as it does not find unsatisfiability ("Solved") in any problem marked satisfiable or countersatisfiable. It also does not solve any unknown or open problems; while this limits the number of new discoveries, any such solutions would need to be checked closely to ensure that they were not caused by an incorrect prover.

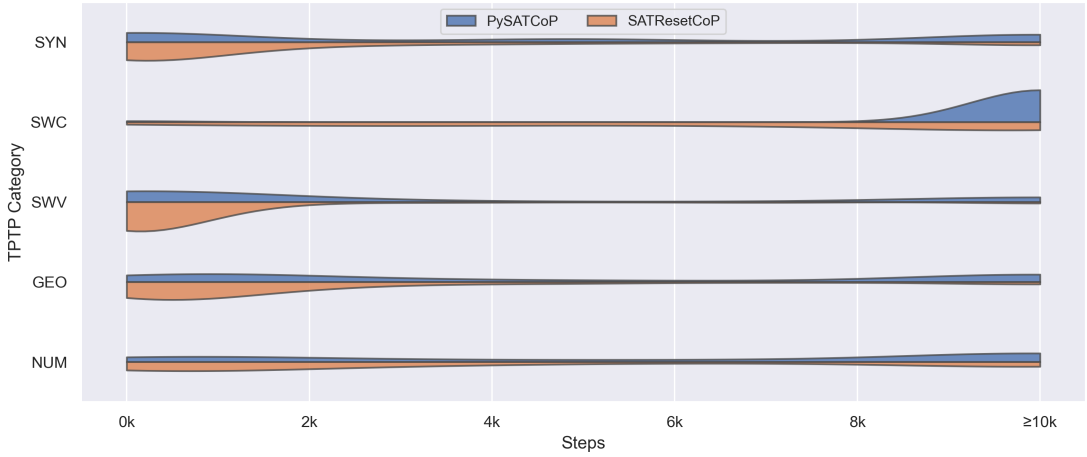


Figure 5: Density of problems solved at each step count by PySATCoP and SATResetCoP on the TPTP categories SYN, SWC, SWV, GEO, and NUM. The area of each violinplot is the same; besides solving more problems in each category, SATResetCoP solves many problems in fewer steps. Unsolved problems are collected in the $\geq 10k$ category on the right.

	FOF		CNF	
Declared status	Unsolved	Solved	Unsolved	Solved
Satisfiable	350	0	1094	0
CounterSatisfiable	732	0	0	0
Unknown	94	0	745	0
Open	4	0	28	0
Unsatisfiable	100	72	4496	1433
ContradictoryAxioms	204	39	0	0
Theorem	4426	1823	0	0

Table 5: Number of problems solved and unsolved by SATResetCoP, grouped by TPTP declared status. No satisfiable or countersatisfiable problem was incorrectly proven to be unsatisfiable.

5 Future Work

SATResetCoP still struggles with a significant number of problems, as seen in Table 3. It would be worth experimenting with adding this strategy to a schedule, following the examples of Otten and Mangla[12, 11], and possibly running a different strategy or solver if no solution is found after a fixed amount of time or number of steps.

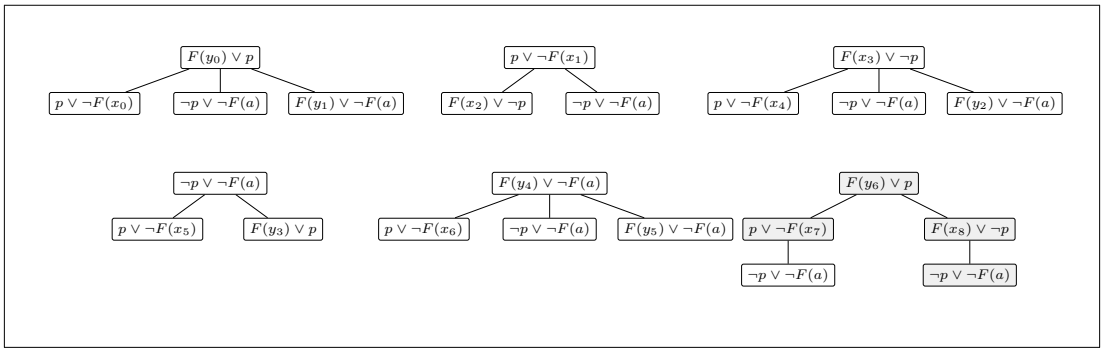
6 Acknowledgements

This work was performed using resources provided by the Cambridge Service for Data Driven Discovery (CSD3) operated by the University of Cambridge Research Computing Service (www.csd3.cam.ac.uk), provided by Dell EMC and Intel using Tier-2 funding from the Engineering and Physical Sciences Research Council (capital grant EP/T022159/1), and DiRAC funding from the Science and Technology Facilities Council (www.dirac.ac.uk).

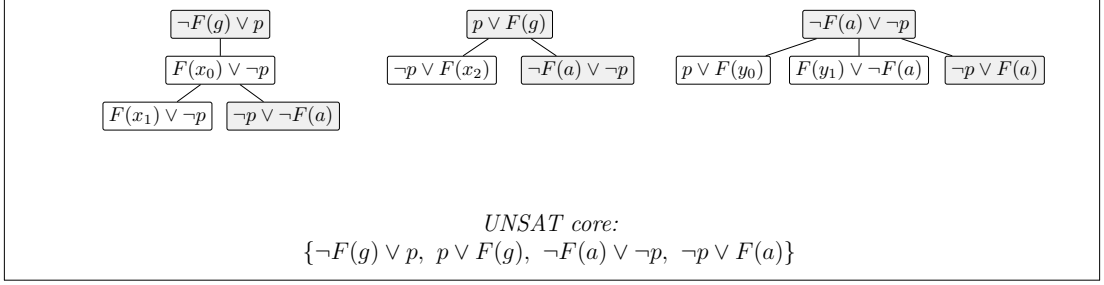
A Solving example

$$p \vee \neg F(X) \qquad F(X) \vee \neg p$$
$$F(Y) \vee p \qquad \neg p \vee \neg F(a) \qquad F(Y) \vee \neg F(a)$$

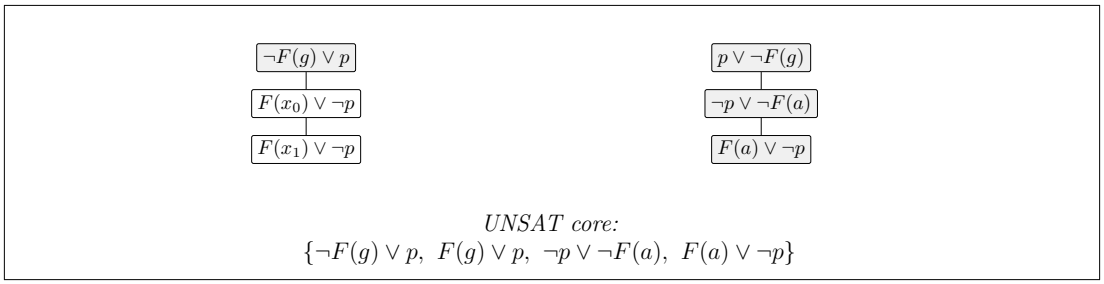
(a) Problem SYN052-1.p[17], adapted from https://tptp.org/TPTP/Problems/SYN/SYN052-1.p.



(b) Solution found by PyLeanCoP in 27 steps.



(c) Solution found by PySATCoP in 19 steps.



(d) Solution found by SATResetCoP in 9 steps.

Figure 6: Unsatisfiability proofs for the TPTP problem SYN052-1[17] produced by three provers: PyLeanCoP, PySATCoP, and SATResetCoP.

B Usage Example

The repo in https://github.com/mfixman/resetsatcop contains two Python files that can be run standalone.

`pycop.py` gets as input a single strategy and a single `.p` file, in either FOF or CNF format. This can be expressed as the path to this file, or a TPTP file specification; in the latter case, the file's path is expected to be inside where the main TPTP directory, which can be specified either with the `-tptp` option or with the `$TPTP` environment variable points.

```
$ python pycop.py --strategy SATResetCoP --print-trace --debug PUZ020-1.p
Start st1: [person(wife)]
Extension ex3: person(wife) -> [-person(X23), -a_truth(statement_by(X23)), knight(X23)] [cut]
Reset 1
Start st4: [-person(X29), knight(X29), knave(X29)]
Extension ex1: -person(X29) -> [person(wife)] [cut]
Extension ex1: knight(X29) -> [-says(X47, Y47), -knight(X47), a_truth(Y47)] [cut]
Reset 1
Start st10: [-a_truth(statement_by(X62)), -person(X62), knight(X62)]
Extension ex5: -a_truth(statement_by(X62)) -> [a_truth(statement_by(husband)), -knight(wife)] [cut]
Start st15: [-knight(husband), knight(wife), -a_truth(statement_by(husband))]
Extension ex1: -knight(husband) -> [-person(X78), knight(X78), -a_truth(statement_by(X78))] [cut]
Extension ex1: knight(wife) -> [a_truth(Y82), -knight(X82), -says(X82, Y82)] [cut]
Reset 1
{'info': 'Theorem (SAT); (-person(husband), -a_truth(statement_by(husband)), knight(husband));
     (a_truth(statement_by(husband)), -knight(wife)); (a_truth(statement_by(husband)), knight(wife));
     (-knight(husband)); (person(husband))', 'steps': 10}
```

`compare_strategies.py` can compare the result of running different strategies in a group of files. The program takes a series of strategies and a directory or set of `.p` problems as an argument (all TPTP problems by default), and returns a CSV with solving information for each strategy/problem pair.

The program aggressively uses multiprocessing, and will try to "fit" each strategy/problem pair into a single processor core. Additionally, it keep partial information of solved problems; runs of `compare_strategies.py -name A` after the first one will retry from where the previous run ended, even if that run crashed or was killed for some reason.

```
$ python compare_strategies.py --name LongTest --strategies SATCoP SATResetCoP $TPTP/Problems/PUZ
Run: partial=partial/LongTest_PUZ  csv=results/LongTest/PUZ.csv  tasks=262  completed=0  remaining=262
     workers=20  timeout=120s
Problem      Strategy     Result
--------------------------------
PUZ004-1.p   SATCoP       0 / 0ms
PUZ004-1.p   SATResetCoP  0 / 0ms
PUZ001-1.p   SATCoP       81 / 17ms
PUZ001-1.p   SATResetCoP  66 / 16ms
[...]

$ head results/LongTest/PUZ.csv | cut -d',' -f1-7 | column -t -s',' -R6,7
problem     strategy     status   result         solved   steps  time_ms
PUZ001+1.p  SATCoP       solved   Theorem (SAT)  True      2076      778
PUZ001+1.p  SATResetCoP  solved   Theorem (SAT)  True       989      821
PUZ001+2.p  SATCoP       timeout                 False    81106   120000
PUZ001+2.p  SATResetCoP  solved   Theorem (SAT)  True     35854    77695
PUZ001-1.p  SATCoP       solved   Theorem (SAT)  True        81       15
PUZ001-1.p  SATResetCoP  solved   Theorem (SAT)  True        66       15
PUZ001-2.p  SATCoP       timeout                 False   497938   120000
PUZ001-2.p  SATResetCoP  timeout                 False   513586   120000
PUZ001-3.p  SATCoP       timeout                 False   245563   120000
```

References

[1] Leo Bachmair, Harald Ganzinger, David McAllester, and Christopher Lynch. Resolution Theorem Proving. In *Handbook of Automated Reasoning*, pages 19–99. Elsevier, 2001.

[2] Bernhard Beckert and Joachim Posegga. leanTAP: Lean tableau-based deduction. *Journal of Automated Reasoning*, 15(3):339–358, 1995.

[3] Evert Willem Beth. *Semantic Entailment and Formal Derivability.* Mededelingen der Koninklijke Nederlandse Akademie van Wetenschappen, Afd. Letterkunde, Nieuwe Reeks. Noord-Hollandsche Uitgevers Maatschappij, 1955. Deel 18, no. 13.

[4] Armin Biere, Tobias Faller, Katalin Fazekas, Mathias Fleury, Nils Froleyks, and Florian Pollitt. CaDiCaL 2.0. In *Computer Aided Verification*, volume 14681 of *Lecture Notes in Computer Science*, pages 133–152. Springer, 2024.

[5] Free Software Foundation. GNU lesser general public license, version 3, 2007.

[6] Michael Färber. A Curiously Effective Backtracking Strategy for Connection Tableaux, 2024.

[7] Michael Färber, Cezary Kaliszyk, and Josef Urban. Machine Learning Guidance for Connection Tableaux. *Journal of Automated Reasoning*, 65(2):287–320, 2021.

[8] Cezary Kaliszyk, Josef Urban, Henryk Michalewski, and Miroslav Olšák. Reinforcement learning of theorem proving. In S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 31, pages 8836–8847. Curran Associates, Inc., 2018.

[9] Reinhold Letz and Gernot Stenz. Model Elimination and Connection Tableau Procedures. In *Handbook of Automated Reasoning*, pages 2015–2114. Elsevier, 2001.

[10] Donald W. Loveland. Mechanical theorem-proving by model elimination. *Journal of the ACM*, 15(2):236–251, 1968.

[11] C. Mangla, S.B. Holden, and L.C. Paulson. Bayesian ranking for strategy scheduling in automated theorem provers. In *International Joint Conference on Automated Reasoning (IJCAR)*, pages 559–577„ 2022.

[12] Jens Otten. Restricting backtracking in connection calculi. *AI Communications*, 23(2–3):159–182, 2010.

[13] Jens Otten. 20 years of leanCoP — an overview of the provers. In *Proceedings of the 1st International Workshop on Automated Reasoning with Connection Calculi (AReCCa@TABLEAUX 2023)*, volume 3613 of *CEUR Workshop Proceedings*, 2023.

[14] Jens Otten and Wolfgang Bibel. leanCoP: Lean connection-based theorem proving. *Journal of Symbolic Computation*, 36(1–2):139–161, 2003.

[15] Jens Otten and Wolfgang Bibel. Advances in Connection-Based Automated Theorem Proving. In Mike Hinchey, Jonathan P. Bowen, and Ernst-Rüdiger Olderog, editors, *Provably Correct Systems*, pages 211–241. Springer International Publishing, 2017.

[16] Jens Otten and Sean Holden. A Syntax for Connection Proofs, 2023.

[17] Francis Jeffry Pelletier. Seventy-five problems for testing automatic theorem provers. *Journal of Automated Reasoning*, 2(2):191–216, 1986.

[18] Michael Rawson, Clemens Eisenhofer, and Laura Kovács. Constraint Learning for Non-confluent Proof Search. In Gian Luca Pozzato and Tarmo Uustalu, editors, *Automated Reasoning with Analytic Tableaux and Related Methods*, volume 15980, pages 103–119. Springer Nature Switzerland, 2026.

[19] Michael Rawson and Giles Reger. Eliminating Models During Model Elimination. In Anupam Das and Sara Negri, editors, *Automated Reasoning with Analytic Tableaux and Related Methods*, volume 12842, pages 250–265. Springer International Publishing, 2021.

[20] Alexandre Riazanov and Andrei Voronkov. The design and implementation of VAMPIRE. *AI Communications*, 15(2–3):91–110, 2002.

[21] Fredrik Rømming, Jens Otten, and Sean B. Holden. Connections: Markov Decision Processes for Classical, Intuitionistic and Modal Connection Calculi. In *Proceedings of the 1st International Workshop on Automated Reasoning with Connection Calculi*, volume 3613 of *CEUR Workshop Proceedings*, pages 107–118, 2023.

[22] Stephan Schulz. Light-weight integration of SAT solving into first-order reasoners: First experiments. In Laura Kovács and Andrei Voronkov, editors, *Vampire 2017. Proceedings of the 4th Vampire Workshop*, volume 53 of *EPiC Series in Computing*, pages 9–19. EasyChair, 2018.

[23] Stephan Schulz and Martin Möhrmann. Performance of Clause Selection Heuristics for Saturation-Based Theorem Proving. In Nicola Olivetti and Ashish Tiwari, editors, *Automated Reasoning*, volume 9706, pages 330–345. Springer International Publishing, 2016.

[24] Geoff Sutcliffe. The TPTP problem library and associated infrastructure: From CNF to TH0, TPTP v6.4.0. *Journal of Automated Reasoning*, 59(4):483–502, 2017.

[25] University of Cambridge Research Computing Services. Ice Lake Nodes. https://docs.hpc.cam.ac.uk/hpc/user-guide/icelake.html, 2026. Accessed 13 May 2026.

[26] Zsolt Zombori, Josef Urban, and Chad E. Brown. Prolog Technology Reinforcement Learning Prover: (System Description). In Nicolas Peltier and Viorica Sofronie-Stokkermans, editors, *Automated Reasoning*, volume 12167, pages 489–507. Springer International Publishing, 2020.

[27] Zsolt Zombori, Josef Urban, and Miroslav Olšák. The Role of Entropy in Guiding a Connection Prover. In Anupam Das and Sara Negri, editors, *Automated Reasoning with Analytic Tableaux and Related Methods*, volume 12842, pages 218–235. Springer International Publishing, 2021.